Wyatt Habinski
400338858
April 9, 2023

Inside the Black Box:

COMPSCI 2DB3

Foreword

As the first step in this solution, I will include any relevant information from the **Description** of the assignment. Most parts are relevant, but some can be omitted. Then, I provide a general analysis highlighting some key insights of the proposed approach. Finally, I answer each of the evaluation questions.

Description

A “transfer” t will be in the form:

transfer($$x$, from, to)

= “transfer object $x from the inventory of account from to the inventory of account to”,that should only be executed if each transfer is possible (the inventory of the from-account has one-or-more copies of the transferred object)

```
UPDATE-INVENTORY(τ, account, object, remove):
Lock_τ(account).
if remove = True then
    Remove one copy of object from the inventory of account.
else
    Add one copy of object to the inventory of account.
end if
Release_τ(account).
```

Figure 1: The pseudo-code for the minimal-locking operation UPDATE-INVENTORY that can add or remove a single object from the inventory of an account.

```
EXECUTE-TRANSACTION(τ):
Commit, Rollback := ∅, ∅.
for each transfer($x, from, to) in τ do
   Lock_τ(from).
   if from holds a copy of $x then
      Remove one copy of object from the inventory of from.
      Lock_τ(to).
      Add one copy of object to the inventory of to.
      Release_τ(to).
      Store the operation "UPDATE-INVENTORY(τ, to, $x, True)" in Rollback.
      Store the operation "UPDATE-INVENTORY(τ, from, $x, False)" in Rollback.
      Release_τ(from).
   else
      Release_τ(from).
      Perform all operations in Rollback.
      return failure.
   end if
end for
return success.
```

Figure 2: The pseudo-code for the transaction execution algorithm.

The following constraints must also never be broken:

C1. No inventory should ever receive a negative number of objects (assuming that all inventories start with a positive number of objects).

C2. As the transfers only move objects between inventories, no objects should be lost or created. Hence, if at any time t no transactions are being executed, then the bag of objects in all inventories at that time t should be equivalent to the initial bag of objects of all inventories.

C3. Successful transactions must have their lasting effects, while failed transactions must not have lasting effects. Hence, if at any time t no transactions are being executed, then the objects in each inventory should reflect the updates due to all transactions that executed successfully before t.

Analysis

In figure 1: The ***UPDATADE-INVENTORY(t, account, object, remove)*** has 2 possible cases. We will evaluate the execution of both cases:

Case 1: When remove = True:

$Lock_t$(account): line 1

$Read_t$(account): read account inventory (line 3)

$Write_t$(account): account := account - object (line 3)

$Release_t$(account): line 7

Case 2: When remove = False:

$Lock_t$(account): line 1

$Read_t$(account): read account inventory (line 5)

$Write_t$(account): account := account + object (line 5)

$Release_t$(account): line 7

In figure 2: The ***EXECUTE-TRANSACTION(t)*** also has 2 possible cases. We will evaluate the execution of both cases:

However, for both cases we have a for loop:

for each <u>transfer</u>($x, from, to) lets call t' in t **do**:

Case 1: When "from" has object "$x"

$Lock_{t'}$(from): line 3

$Read_{t'}$(from): **if** statement read "from" inventory (line 4)

$Write_{t'}$(from): from := from - $x (line 5)

$Lock_{t'}$(to): line 6

$Read_{t'}$(to): read "to" inventory (line 7)

$Write_{t'}$(to): to := to + $x (line 7)

$Release_{t'}$(to): line 8

– Store ***UPDATE-INVENTORY(t', to, $x, True)*** in Rollback: line 9

– Store ***UPDATE-INVENTORY(t', from, $x, False)*** in Rollback: line 10

$Release_{t'}$(from): line 11

Case 2: When "from" does not have $x

$Lock_{t'}$(from): line 3

$Read_{t'}$(from): **if** statement line 4

$Release_{t'}$(from): line 13

– Perform all operations in Rollback

Evaluation

Q1: Does the proposed approach follow strict two-phase locking? Does it follow two-phase locking?

The proposed solution does not follow strict two-phase locking and also does not follow two-phase locking. To see this we consider the execution of a transaction:

t = transfer(Apple, Alicia, Bo), transfer(Pear, Eva, Celeste))

The database used in the example:

Alicia	Apple
Bo	None
Eva	Banana
Celeste	None

The basic operations performed by the transaction t, when executed on the provided database:

For loop of EXECUTE-TRANSACTION(t)
First transaction in t: transfer(Apple, Alicia, Bo)
$Lock_t$(Alicia)
$Read_t$(Alicia)
$Write_t$(Alicia)
$Lock_t$(Bo)
$Read_t$(Bo)
$Write_t$(Bo)

$Release_t$(Bo)
Store UPDATE-INVENTORY(t, Bo, Apple, True) in Rollback
Store UPDATE-INVENTORY(t, Alicia, Apple, False) in Rollback
$Release_t$(Alicia)
Second transaction in t: <u>transfer</u>(Pear, Eva, Celeste)
$Lock_t$(Eva)
$Read_t$(Eva)
$Release_t$(Eva)
Execute UPDATE-INVENTORY(t,Bo,Apple,True)
$Lock_t$(Bo)
$Read_t$(Bo)
$Write_t$(Bo)
$Release_t$(Bo)
Execute UPDATE-INVENTORY(t,Alicia,Apple,False)
$Lock_t$(Alicia)
$Read_t$(Alicia)
$Write_t$(Alicia)
$Release_t$(Alicia)

If we look at the execution of t via EXECUTE-TRANSACTION(t) in isolation on this database, then it will perform the operations outlined above. As is clear from the steps above and the highlighted rows, the execution obtains a lock after it has already released locks. This is in violation of the restrictions placed on executions of transactions by the two-phase locking and strict-two phase locking protocols.

Q2 Does the proposed approach suffer from deadlocks?

Yes, the proposed approach could suffer from deadlocks. A deadlock occurs when a transaction tries to obtain a lock while already holding a lock. In the proposed solution, mainly the sudo code for EXECUTE-TRANSACTION, we have that on line 3, we $Lock_t$(from) and if “from” has a

sufficient inventory, we $Lock_t$(to) on line 6 without ever releasing “from”. Therefore we have a transaction that holds two locks at the same time, and will suffer from deadlocks.

Q3 Does the proposed approach expose read-write, write-read, and or write-write conflicts?

To show that the approach has read-write and write-read conflicts, we consider the following two transactions:

t1 = (<u>transfer</u>(Apple,Alicia, Bo), <u>transfer</u>(Pear, Eve, Celeste))

t2 = (<u>transfer</u>(Apple,Alicia, Eve)

and we consider the following valid interleaved execution of t1 and t2, that follows from the proposed approach:

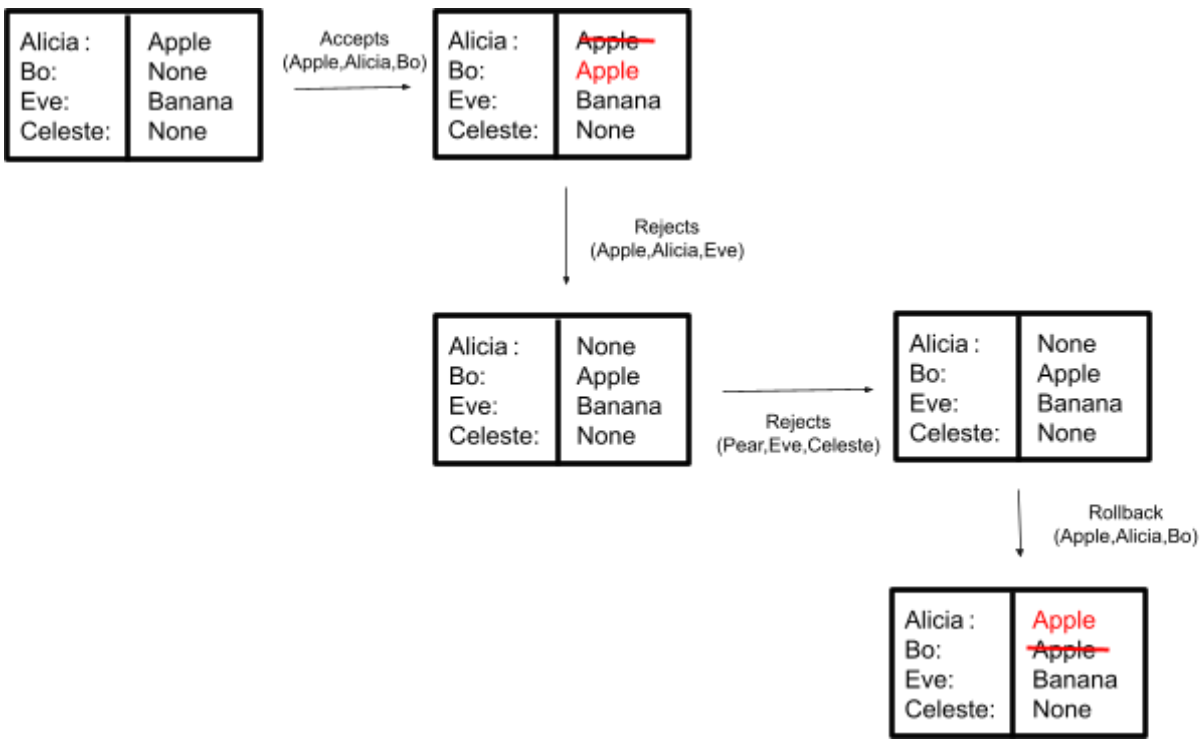


figure 1

The interleaved execution yields the schedule:

t1	t2

$Lock_{t1}$(Alicia)	
$Read_{t1}$(Alicia)	
$Write_{t1}$(Alicia)	
$Lock_{t1}$(Bo)	
$Read_{t1}$(Bo)	
$Write_{t1}$(Bo)	
$Release_{t1}$(Bo)	
$Release_{t1}$(Alicia)	
	$Lock_{t2}$(Alicia)
	$Read_{t2}$(Alicia)
	$Release_{t2}$(Alicia)
$Lock_{t1}$(Eve)	
$Read_{t1}$(Eve)	
$Release_{t1}$(Eve)	
- Perform Rollbacks	
$Lock_{t1}$(Bo)	
$Read_{t1}$(Bo)	
$Write_{t1}$(Bo)	
$Release_{t1}$(Bo)	
$Lock_{t1}$(Alicia)	
$Read_{t1}$(Alicia)	
$Write_{t1}$(Alicia)	
$Release_{t1}$(Alicia)	
Abort t1	Abort t2

table 1

Write-Read Conflict: In this execution t2, reads uncommitted value from Alicia (green highlight) that was previously written by t1 (blue highlight) which is a write-read conflict

Read-Write Conflict: In this execution t1 writes a value to Alicia (red highlight) that was previously read by t2 (green highlight)

To show that the approach has write-write conflicts, we consider the following valid interleaved execution of the following new transactions:

t1 = (transfer(Apple,Alicia, Bo), transfer(Pear, Eve, Celeste))

t2 = (transfer(Apple,Alicia, Celeste)

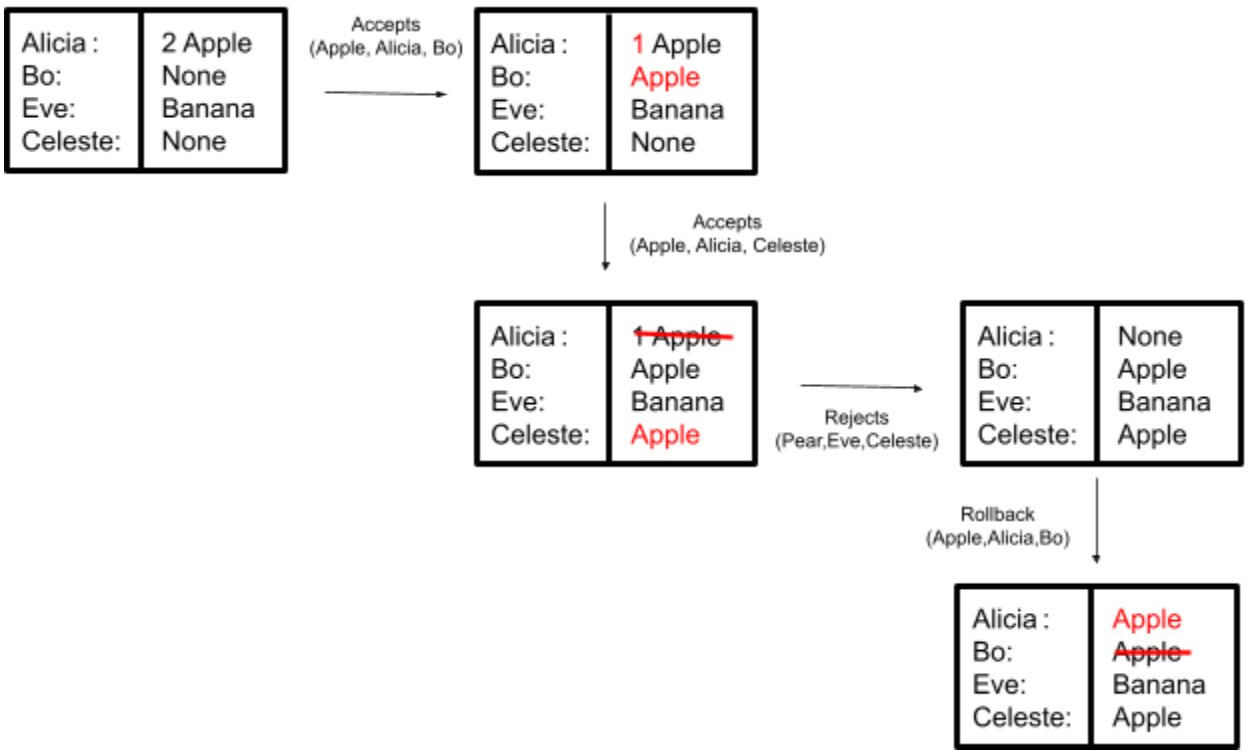


figure 2

t1	t2
$Lock_{t1}$(Alicia)	
$Read_{t1}$(Alicia)	
$Write_{t1}$(Alicia)	
$Lock_{t1}$(Bo)	
$Read_{t1}$(Bo)	

$Write_{t1}$(Bo)	
$Release_{t1}$(Bo)	
$Release_{t1}$(Alicia)	
	$Lock_{t2}$(Alicia)
	$Read_{t2}$(Alicia)
	$Write_{t2}$(Alicia)
	$Lock_{t2}$(Celeste)
	$Read_{t2}$(Celeste)
	$Write_{t2}$(Celeste)
	$Release_{t2}$(Celeste)
	$Release_{t2}$(Alicia)
$Lock_{t1}$(Eve)	
$Read_{t1}$(Eve)	
$Release_{t1}$(Eve)	
- Perform Rollbacks	
$Lock_{t1}$(Bo)	
$Read_{t1}$(Bo)	
$Write_{t1}$(Bo)	
$Release_{t1}$(Bo)	
$Lock_{t1}$(Alicia)	
$Read_{t1}$(Alicia)	
$Write_{t1}$(Alicia)	
$Release_{t1}$(Alicia)	
Abort t1	Commit t2

table 2

Write-Write Conflict: In this execution, t2 reads and writes an uncommitted value from Alicia (red highlight) that was previously written by t1 (green highlight) which is a write-write conflict.

Q4 Does the proposed approach suffer from dirty reads?

The approach does have dirty reads. We can refer to the first part of question 3, where we use the following transactions:

t1 = (transfer(Apple,Alicia, Bo), transfer(Pear, Eve, Celeste))

t2 = (transfer(Apple,Alicia, Eve)

Again using the same figure (figure 1) and table (table 1), we can see that t2 reads Alicia after t1 already wrote to Alicia. However, the initial write from t1 is never committed and therefore t2 reads an uncommitted value. Hence, a dirty read.

Q5 Does the proposed approach suffer from unrepeatable reads?

The proposed approach suffers from unrepeatable reads. We refer to the second part of question 3, where we use the following transactions:

t1 = (transfer(Apple,Alicia, Bo), transfer(Pear, Eve, Celeste))

t2 = (transfer(Apple,Alicia, Celeste)

Again, we will use the respective figure (figure 2) and table (table 2). We see that the first transfer in t1 reads and writes the value in Alicia from 2 Apples to 1 Apple. On the following subsequent read of t1 during the second transfer, it reads the value of Alicia as “None”. Hence this repeated read has a different value than previously established by t1 and therefore the approach suffers from unrepeatable reads.

Q6 Is the proposed approach serializable?

The approach is not serializable considering we have already seen that it suffers from both dirty reads and unrepeatable reads.

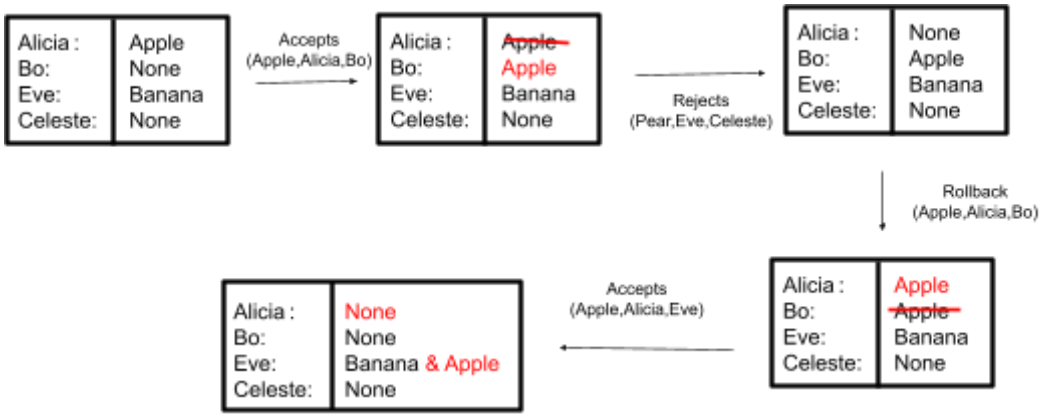
Alternatively, we can show a transaction that is not equivalent to any serial execution. The transactions used in the first part of question 3:

t1 = (transfer(Apple,Alicia, Bo), transfer(Pear, Eve, Celeste))

t2 = (transfer(Apple,Alicia, Eve)

Have only 2 serial execution schedules. This is when t1 is fully executed before t2 is fully executed, or when t2 is fully executed before t1. I will show the database states during execution when performed on the same initial database of these two executions.

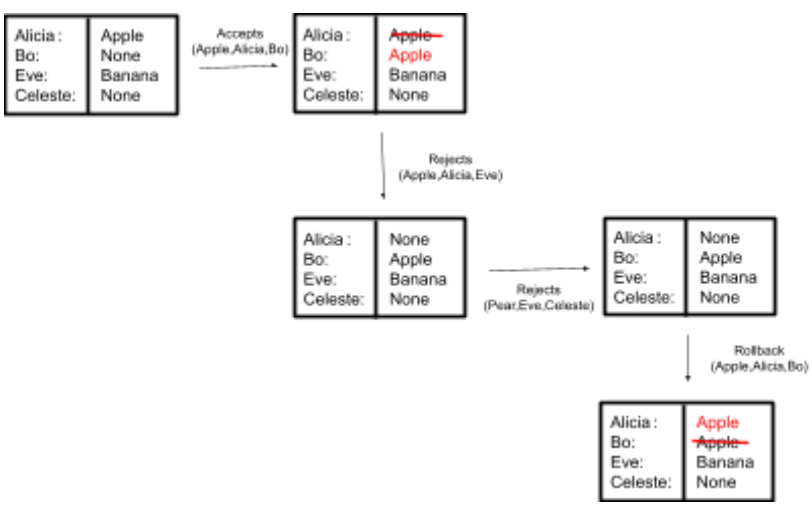
t1, then t2:



t2, then t1:

As we can see the final state in both execution paths are the same. However when we consider an interleaved execution; the same interleaved schedule as used in figure 1 and table 1 of question 3, we have a different final state. Hence, the proposed approach is not serializable.

(Recited from figure 1 in Q3) Interleaved:



Q7 Are the constraints C1-C3 as set out by the game developer satisfied?

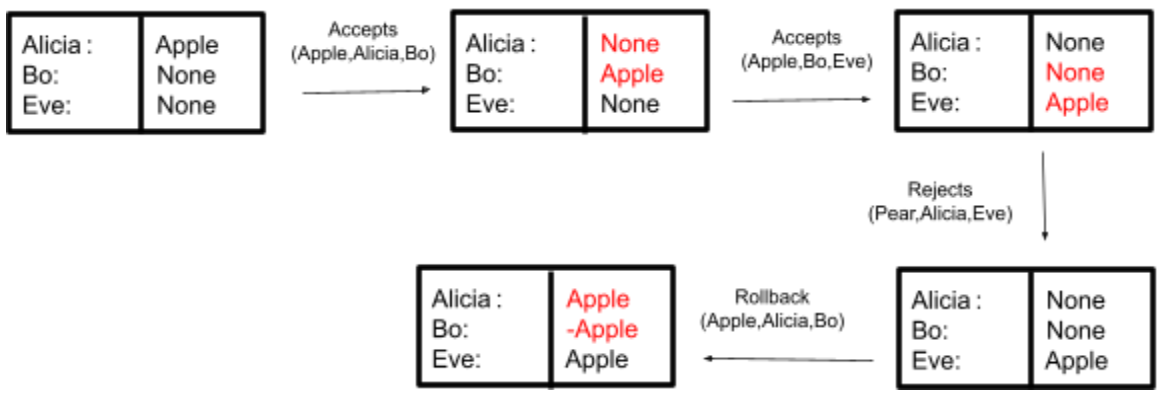
C1) This constant does <u>not</u> hold. This can be shown in the following two transactions:

t1 = (<u>transfer</u>(Apple, Alicia, Bo), <u>transfer</u>(Pear, Alicia, Eve))

t2 = (<u>transfer</u>(Apple, Bo, Eve))

This constant will not hold during interleaved transactions. For example, when an account “to” which initially starts with 0 amount of an object, receives an object from transaction 1, then because it’s interleaved with transaction 2, it transfers that object away (in transaction 2). Then transaction 2 is completed and committed, but transaction 1 fails the following transfer resulting in a rollback of the first initial transfer. This means the account “to” would start with 0, receive 1 to have a total of 1. Then give 1 away for a total of 0. Then have the first +1 rolled back effectively taking 1 away from its total of 0 which leaves it in a negative state.

This is exemplified in the following schedule using t1 and t2 stated above:



C2) This constraint holds. An object is removed and added in only 2 ways. The first is when there is no issue with the transaction. In this case, an object is removed from the “from” account, and subsequently, added to the “t” account via lines 5 and 7 in EXECUTE-TRANSACTION. The other case is when a rollback is performed. However, operations in rollback are only ever added in pairs of “addition of object to inventory to” and “removal of same object to inventory from”. Hence whenever rollback is called, the operations cancel eachother out. Each removal of an object is accompanied by an addition of an object. Therefore either only both addition and removal of an object are successfully committed, or both are negated.

C3) This constraint holds. Successful transactions performs inventory removals of $x object from accounts “from” in the EXECUTE-TRANSACTION operation and for each such removal, it follows that there is an addition of $x object to the “to” inventory also via EXECUTE-TRANSACTION. The combination of such a removal followed directly by an addition corresponds to the execution of a single successful <u>transfer</u>($x,from,to) clause which is a lasting change.

Failed transactions have two possibilities. Case one, no object is added or removed to an inventory. This takes effect in EXECUTE-TRANSACTION due to the if else statement. However if a failed transaction can have transfers that eventually get rolled back. In this case, inventory object removals take place via the rollback in EXECUTE-TRANSACTION. The rollback calls operation UPDATE-INVENTORY for both the removal and addition of an object in succession. This operation will restore the removed object to the correct inventory. The combination of such a removal and addition operation assures that no lasting inventory balance is made by failed transactions.